REPUBLIQUE DU CAMEROUN
****
PAIX-TRAVAL-PATRIE
****
UNIVERSITE DE DSCHAN
****
FACULTES DES SCIENCES
****
DEPARTEMENT DE SCIENCES

REPUBLIC OF CAMEROON
****
PEACE-WORK-FATHERLAND
****
UNIVERSITY OF DSCHANG
****
FACULTY OF SCIENCES
****
DEPARTEMENT OF SCIENCES

IAR418 - Fouille de Données

CHAPITRE I : Introduction au Data Mining

Présenté par :

DNJOMOU YONMBA WILFRIED LOIC	**CM-UDS-24SCI0999**
KENFACK LEONEL	**CM-UDS-17SCI0998**
SAGUEU WAKAM DILANE	**CM-UDS-24SCI1040**

Superviseur : Pr. KENGNE TCHENDJI Vianney

Année académique 2024-2025

Table des matières

Introduction générale **3**

1 Combinaison de classifieurs **4**

1.1 Définition et phases de la combinaison de classifieurs 4

1.1.1 Nature des sorties et niveaux d'information 5

1.2 Motivations théoriques et justifications statistiques 5

1.2.1 Théorème du jury de Condorcet 6

1.2.2 Décomposition biais–variance 6

1.2.3 Théorie unifiée de la diversité dans les ensembles 6

1.2.4 Les problèmes de Dietterich 8

1.3 Typologies des méthodes d'ensemble 8

1.3.1 Bagging (bootstrap aggregating) 8

1.3.2 Boosting 12

1.3.3 Stacking (stacked generalization) 15

1.3.4 Voting (vote majoritaire vs vote probabiliste) 15

1.4 Analyse comparative des approches 16

1.5 Aspects architecturaux des systèmes multi-classifieurs 17

1.5.1 Architectures de contrôle de flux 17

1.5.2 Systèmes de sélection dynamique (DS) 18

1.6 Avantages et limites de la combinaison 18

1.6.1 Avantages 18

1.6.2 Limites 18

2 Combinaison de classifieurs et explicabilité **19**

2.1 Le Problème de la Boîte Noire 20

2.1.1 Qu'est-ce qu'une boîte noire en Machine Learning ? 20

2.1.2 Comment l'agrégation crée-t-elle l'opacité ? 20

2.1.3 Pourquoi l'explicabilité est-elle indispensable ? 21

2.2 Complexité des Ensembles de Modèles 22

2.2.1 La dimension VC : mesurer la capacité d'un modèle 22

2.2.2 La borne de généralisation : pourquoi la complexité nuit à l'explication . . 22

2.2.3 Bagging et Boosting : deux architectures, deux types de complexité 23

2.3 Stratégies d'Explicabilité des Ensembles (XAI) 24

2.3.1 Les valeurs SHAP : une explication fondée sur la théorie des jeux 24

2.3.2 LIME : une explication locale par approximation linéaire 26
2.3.3 Extraction de règles globales : de la forêt aux règles médicales (SIRUS) . . 27
2.3.4 Distillation de connaissances : enseigner la transparence 28
2.4 Compromis Performance–Compréhension 29
2.4.1 Le front de Pareto : visualiser le compromis 29
2.4.2 Quantifier l'interprétabilité : les métriques de fidélité et stabilité 29
2.4.3 Les modèles hybrides : fusionner performance et transparence 30
2.4.4 Les EBM : des ensembles intrinsèquement transparents 31
2.4.5 Cas fil rouge : atteindre l'optimum de Pareto pour le déploiement CKD . 31

Conclusion générale **34**

Introduction générale

L'intelligence artificielle et l'apprentissage automatique occupent une place croissante dans les systèmes de décision, industriels, médicaux, financiers ou réglementaires. La **combinaison de classifieurs**, ou apprentissage d'ensemble (*ensemble learning*), s'impose comme un paradigme central : on construit un comité de modèles dont les prédictions sont agrégées pour améliorer la précision et la robustesse (bagging, boosting, stacking). Parallèlement, l'**explicabilité des modèles** est devenue une exigence majeure : les décisions automatisées sont scrutées par les utilisateurs, les régulateurs et la société ; comprendre *pourquoi* un modèle prédit relève de la confiance, de la conformité et de l'éthique. Les méthodes d'explicabilité (intrinsèques ou post-hoc : LIME, SHAP, importance des variables) visent à rendre les systèmes moins opaques. Ce rapport se situe à l'intersection de ces deux thématiques : les systèmes d'ensemble complexifient le modèle global et renforcent l'effet « boîte noire », tout en offrant des gains de performance substantiels.

Les méthodes d'ensemble améliorent la précision au prix d'une complexité accrue (plusieurs modèles, fusion, dépendances), ce qui soulève une question centrale : *comment concilier la puissance des systèmes combinant plusieurs classifieurs avec l'exigence d'explicabilité et d'interprétabilité des décisions ?* D'un côté, un classifieur unique peut être plus facile à expliquer ; de l'autre, un ensemble peut être plus performant mais plus difficile à interpréter. La problématique recouvre les dimensions techniques, architecturales et réglementaires (transparence, traçabilité).

Les objectifs sont de présenter les fondements de la combinaison de classifieurs (définitions, typologies, motivations théoriques, aspects architecturaux) ; de clarifier les concepts d'explicabilité et d'interprétabilité et les principales méthodes (intrinsèques et post-hoc : LIME, SHAP, PDP/ICE, explicabilité locale et globale) ; d'analyser les enjeux spécifiques aux ensembles (complexité, stratégies d'explication, compromis performance et compréhension) ; et de mettre en perspective les avantages et limites au regard de l'explicabilité.

Le document comporte trois parties : la **combinaison de classifieurs** (définition, typologies, aspects architecturaux, analyse comparative, avantages et limites) ; l'**explicabilité des modèles** (définition, interprétabilité, méthodes, enjeux dans les systèmes décisionnels, éthique et réglementation) ; la **combinaison et l'explicabilité** (boîte noire, stratégies, compromis performance et compréhension). La conclusion générale synthétise les apports et ouvre sur les perspectives.

Chapitre 1

Combinaison de classifieurs

Introduction

L'évolution de l'apprentissage automatique a franchi une étape décisive avec l'émergence des systèmes de classifieurs multiples (*Multiple Classifier Systems – MCS*), plus communément désignés sous le terme d'apprentissage d'ensemble [10, 32, 18]. Cette approche repose sur un paradigme fondamental : la combinaison stratégique de plusieurs modèles de prédiction pour pallier les insuffisances inhérentes à tout classifieur unique.

Historiquement, la recherche en intelligence artificielle s'est concentrée sur l'optimisation d'algorithmes individuels. Cependant, l'expérience a démontré qu'un modèle unique, aussi sophistiqué soit-il, est souvent limité par ses propres biais structurels et sa sensibilité au bruit des données. L'apprentissage d'ensemble propose de substituer au modèle solitaire un comité de décideurs dont la sagesse collective surpasse les capacités individuelles.

Cette section propose une exploration de la combinaison de classifieurs, structurée autour des fondements théoriques, des typologies méthodologiques et des considérations architecturales qui définissent ce domaine. L'analyse s'attache à montrer comment la diversité des modèles et la rigueur de leur agrégation permettent d'atteindre des niveaux de performance et de robustesse inaccessibles aux approches classiques.

1.1 Définition et phases de la combinaison de classifieurs

La **combinaison de classifieurs** est définie comme le processus d'intégration des sorties de plusieurs modèles d'apprentissage afin de générer une décision globale plus fiable et précise. Un système multi-classifieurs est structurellement composé d'un ensemble d'apprenants de base, souvent qualifiés d'*apprenants faibles* (*weak learners*) lorsqu'ils sont pris isolément, et d'un mécanisme de fusion qui orchestre leurs réponses.

Le cycle de vie d'un tel système se décompose généralement en trois phases critiques :

1. **Génération de l'ensemble** : cette phase initiale consiste à produire un pool de modèles de base. L'objectif est de créer des modèles qui ne sont pas seulement performants, mais surtout complémentaires. La complémentarité est assurée par l'introduction de diversité, que ce soit par l'utilisation de différents sous-ensembles de données, de paramètres variés pour un même algorithme, ou de familles d'algorithmes distinctes.

2. **Sélection ou élagage (*pruning*)** : une fois le pool généré, il peut être nécessaire de réduire la taille de l'ensemble. L'élagage vise à éliminer les modèles redondants ou ceux dont la performance dégrade la décision collective. Cette étape permet non seulement d'améliorer la précision globale mais aussi de réduire les coûts computationnels lors de l'inférence.
3. **Combinaison ou fusion** : la phase finale consiste à fusionner les prédictions des modèles sélectionnés pour obtenir la réponse ultime. Le choix de la fonction de combinaison dépend crucialement du type d'information fournie par les classifieurs de base (niveau d'information en sortie).

1.1.1 Nature des sorties et niveaux d'information

L'efficacité de la combinaison dépend de la richesse des informations transmises par les modèles individuels. On distingue classiquement trois niveaux de sortie :

— **Niveau « Classe » (hard output)** : le classifieur fournit uniquement l'étiquette de la classe prédite. C'est l'information la plus élémentaire, où chaque modèle agit comme un votant binaire.

— **Niveau « Rang »** : le classifieur produit une liste ordonnée des classes possibles, de la plus probable à la moins probable. Ce format est particulièrement utile lorsque la classe réelle ne figure pas au premier rang mais reste parmi les premières propositions.

— **Niveau « Mesure » (soft output)** : c'est le niveau le plus riche. Le classifieur associe à chaque classe une valeur numérique, telle qu'une probabilité ou un score de confiance. Cette mesure quantifie l'incertitude du modèle et permet des fusions pondérées beaucoup plus fines.

Le tableau suivant présente une comparaison structurelle de ces types d'informations en sortie.

TABLE 1.1 – Comparaison des niveaux d'information en sortie des classifieurs

Type de sortie	Information contenue	Complexité de fusion	Cas d'utilisation typique
Classe	Étiquette discrète uniquement	Faible	Vote majoritaire simple
Rang	Préférences relatives ordonnées	Moyenne	Méthode de Borda, systèmes de recommandation
Mesure	Probabilités, scores de confiance	Élevée	Moyenne pondérée, stacking, modèles bayésiens

1.2 Motivations théoriques et justifications statistiques

La supériorité des systèmes multi-classifieurs n'est pas une simple observation empirique ; elle s'appuie sur des piliers théoriques robustes, notamment le théorème du jury de Condorcet et la décomposition biais–variance [29, 1, 13].

1.2.1 Théorème du jury de Condorcet

Formulé à l'origine dans le contexte des sciences sociales, ce théorème constitue une base théorique fondamentale pour l'apprentissage d'ensemble [33, 6]. Il postule que si un groupe d'individus doit prendre une décision binaire (par exemple, classer un objet comme A ou B) et que chaque individu a une probabilité p supérieure à 0,5 de choisir la bonne réponse, alors la probabilité que la décision de la majorité soit correcte augmente avec le nombre d'individus.

Mathématiquement, si L est le nombre de classifieurs indépendants et p leur précision individuelle, la probabilité P que l'ensemble prenne la bonne décision par vote majoritaire tend vers 1 lorsque L tend vers l'infini. À l'inverse, si $p < 0{,}5$, l'ajout de classifieurs réduit la performance globale. Cette démonstration justifie l'utilisation d'« apprenants faibles » pour construire des systèmes puissants, à condition qu'ils soient plus performants qu'un tirage aléatoire et qu'ils soient suffisamment indépendants.

1.2.2 Décomposition biais–variance

D'un point de vue statistique, l'erreur de généralisation d'un modèle d'apprentissage peut être décomposée en trois composantes : le biais au carré, la variance et l'erreur irréductible (bruit) [31, 15]. La formule de l'erreur quadratique moyenne (MSE) est :

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathrm{Bias}\big[\hat{f}(x)\big]^2 + \mathrm{Var}\big[\hat{f}(x)\big] + \sigma^2, \tag{1.1}$$

où σ^2 représente le bruit inhérent aux données.

— Le **biais** mesure l'écart entre la prédiction moyenne du modèle et la valeur réelle. Un biais élevé est le signe d'un modèle trop simpliste qui ne capture pas la complexité des données (sous-apprentissage, *underfitting*).

— La **variance** mesure la dispersion des prédictions pour différents jeux de données d'apprentissage. Une variance élevée indique que le modèle est trop sensible aux fluctuations et au bruit des données (sur-apprentissage, *overfitting*).

L'apprentissage d'ensemble permet de naviguer dans ce compromis biais–variance. Les méthodes de type bagging agissent principalement en réduisant la variance par la moyenne de modèles complexes, tandis que les méthodes de boosting se concentrent sur la réduction du biais en agrégeant itérativement des modèles simples pour corriger leurs erreurs systématiques.

1.2.3 Théorie unifiée de la diversité dans les ensembles

Une contribution théorique importante aux méthodes d'ensemble consiste à faire apparaître explicitement la diversité comme composante de la décomposition de la perte. Pour une vaste famille de fonctions de perte (notamment les divergences de Bregman, qui incluent l'erreur quadratique moyenne et l'entropie croisée), on peut montrer que la perte de l'ensemble s'exprime comme la moyenne des pertes individuelles diminuée d'un terme de diversité [24].

Dans le cas particulier de l'erreur quadratique moyenne, cette idée est capturée par l'identité de Krogh et Vedelsby [23]. Pour un ensemble de M modèles dont les prédictions sont

$q_1(x), \ldots, q_M(x)$ pour une entrée x et une cible y, on pose

$$\bar{q}(x) = \frac{1}{M} \sum_{i=1}^{M} q_i(x).$$

L'identité s'écrit alors :

$$\big(\bar{q}(x) - y\big)^2 = \frac{1}{M} \sum_{i=1}^{M} \big(q_i(x) - y\big)^2 - \frac{1}{M} \sum_{i=1}^{M} \big(q_i(x) - \bar{q}(x)\big)^2. \tag{1.2}$$

Le membre de gauche est l'erreur quadratique de l'ensemble, le premier terme à droite la moyenne des erreurs quadratiques individuelles, et le second terme (souvent appelé « ambiguïté » ou « diversité ») mesure le degré de désaccord entre les prédictions des modèles. L'identité montre que, toutes choses égales par ailleurs, l'erreur de l'ensemble diminue lorsque la diversité augmente.

Démonstration de l'identité de Krogh et Vedelsby. Pour simplifier les notations, on note $\bar{q} = \bar{q}(x)$ et $q_i = q_i(x)$.

(i) Développement de l'erreur de l'ensemble.

$$(\bar{q} - y)^2 = \bar{q}^2 - 2\bar{q}y + y^2.$$

(ii) Développement de la moyenne des erreurs individuelles.

$$\begin{aligned}
\frac{1}{M} \sum_{i=1}^{M} (q_i - y)^2 &= \frac{1}{M} \sum_{i=1}^{M} (q_i^2 - 2q_i y + y^2) && (1.3)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2y \left(\frac{1}{M} \sum_{i=1}^{M} q_i \right) + y^2 && (1.4)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2. && (1.5)
\end{aligned}$$

(iii) Développement du terme de diversité.

$$\begin{aligned}
\frac{1}{M} \sum_{i=1}^{M} (q_i - \bar{q})^2 &= \frac{1}{M} \sum_{i=1}^{M} (q_i^2 - 2q_i\bar{q} + \bar{q}^2) && (1.6)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2\bar{q} \left(\frac{1}{M} \sum_{i=1}^{M} q_i \right) + \bar{q}^2 && (1.7)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - 2\bar{q}^2 + \bar{q}^2 && (1.8)\\
&= \frac{1}{M} \sum_{i=1}^{M} q_i^2 - \bar{q}^2. && (1.9)
\end{aligned}$$

(iv) Soustraction des deux termes.

$$\frac{1}{M}\sum_{i=1}^{M}(q_i - y)^2 - \frac{1}{M}\sum_{i=1}^{M}(q_i - \bar{q})^2 = \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2\right) - \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - \bar{q}^2\right) \tag{1.10}$$

$$= -2y\bar{q} + y^2 + \bar{q}^2 \tag{1.11}$$

$$= (\bar{q} - y)^2. \tag{1.12}$$

On retrouve bien l'identité annoncée. Comme le terme de diversité est une somme de carrés, il est toujours positif ou nul. L'erreur de l'ensemble est donc toujours inférieure ou égale à la moyenne des erreurs individuelles, avec égalité lorsque tous les modèles prédisent la même valeur (diversité nulle).

Cette identité justifie l'importance de promouvoir la diversité dans les ensembles (via le bagging, le boosting, la randomisation des architectures, etc.) et ouvre sur des généralisations à d'autres pertes au moyen des divergences de Bregman [24].

1.2.4 Les problèmes de Dietterich

Thomas Dietterich a identifié trois raisons fondamentales expliquant pourquoi un ensemble peut surpasser un modèle unique :

1. **Le problème statistique** : lorsque le volume de données est faible par rapport à la taille de l'espace des hypothèses, l'algorithme peut trouver plusieurs modèles qui s'ajustent parfaitement aux données d'entraînement. En les combinant, l'ensemble réduit le risque de choisir par erreur un modèle qui généralise mal.
2. **Le problème computationnel** : de nombreux algorithmes (comme les réseaux de neurones) utilisent des heuristiques de recherche locale qui peuvent rester bloquées dans des optima locaux. Un ensemble effectuant plusieurs recherches à partir de points de départ différents a plus de chances de trouver une meilleure approximation globale.
3. **Le problème de représentation** : parfois, la véritable fonction cible ne peut être représentée par aucun des modèles de l'espace d'hypothèses choisi. La combinaison de plusieurs modèles permet d'étendre cet espace et d'approcher des fonctions plus complexes.

1.3 Typologies des méthodes d'ensemble

Les méthodes d'ensemble se divisent principalement en approches parallèles (bagging), séquentielles (boosting) et hiérarchiques (stacking) [3, 34, 26].

1.3.1 Bagging (bootstrap aggregating)

Le bagging (pour *bootstrap aggregating*) vise à réduire la variance en entraînant plusieurs modèles indépendants en parallèle sur des échantillons de données légèrement différents. Introduit par Breiman [2] et largement diffusé ensuite [19, 16, 1], il est particulièrement efficace pour des apprenants de base instables comme les arbres de décision profonds.

Définition. À partir d'un ensemble d'apprentissage $D = \{(x_j, y_j)\}_{j=1}^n$, le bagging construit m échantillons bootstrap D_i par tirage aléatoire avec remise, chacun de taille n. Sur chaque D_i, on entraîne un modèle de base $\hat{f}_i$ (*base learner*). À la prédiction, les sorties des m modèles sont agrégées :

— en régression, par moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x);$$

— en classification, par vote majoritaire

$$\hat{y}(x) = \arg\max_k \sum_{i=1}^{m} \mathbb{I}\big(\hat{y}_i(x) = k\big).$$

La construction des m modèles étant indépendante, l'entraînement est naturellement parallélisable et s'adapte bien aux architectures multi-cœurs ou distribuées [3, 19].

Fonctionnement détaillé. On peut résumer la procédure de bagging en quatre étapes [2, 25] :

1. **Choix de l'apprenant de base et du nombre de modèles** : on fixe un modèle de base (par exemple, un arbre de décision) et un nombre de réplications m (souvent entre 50 et 500).
2. **Génération des échantillons bootstrap** : pour chaque $i \in \{1, \ldots, m\}$, on tire n exemples de D avec remise pour former D_i. Chaque observation a une probabilité $1/n$ d'être choisie à chaque tirage ; au total, environ 63,2 % des exemples distincts apparaissent dans D_i, les autres constituant les données *out-of-bag*.
3. **Entraînement des modèles** : on entraîne indépendamment un modèle $\hat{f}_i$ sur chaque D_i. Cette étape est triviale à paralléliser.
4. **Agrégation des prédictions** : pour une nouvelle entrée x, on calcule les prédictions des m modèles et on les agrège par moyenne (régression) ou vote (classification), comme indiqué ci-dessus.

Un pseudocode simplifié est le suivant :

```
fonction Bagging(D, m, base_learner):
    modèles = []
    pour i de 1 à m:
        D_i = bootstrap_sample(D)      // tirage avec remise
        f_i = base_learner.fit(D_i)
        modèles.ajouter(f_i)
    retour modèles

fonction Predire(modèles, x):
    prédictions = [f_i.predict(x) pour f_i dans modèles]
    si régression:      retour moyenne(prédictions)
    si classification:  retour mode(prédictions)
```

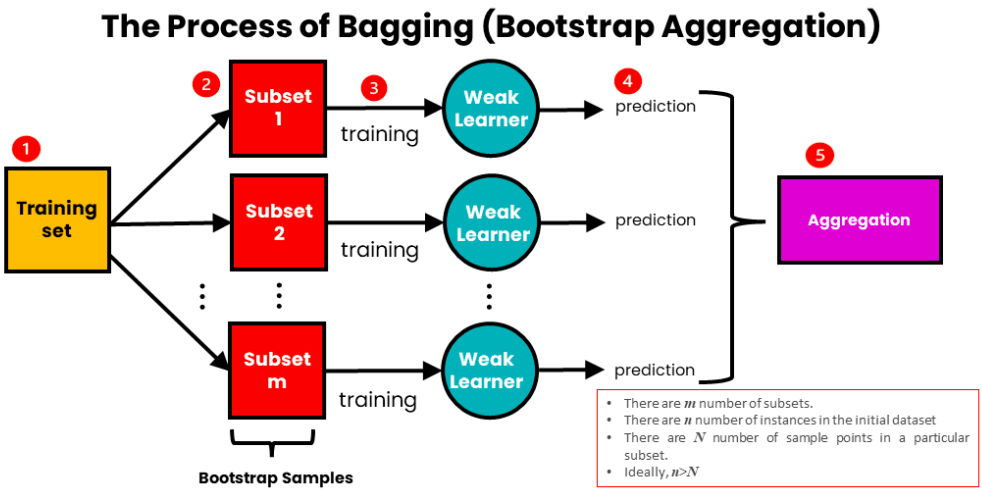

FIGURE 1.1 – Schéma pédagogique du bagging : du jeu de données original aux échantillons bootstrap, puis aux modèles de base et à l'agrégation.

Rôle des données out-of-bag (OOB). Lors de la génération de D_i, chaque exemple de D a une probabilité $(1 - 1/n)^n \simeq 1/e$ de ne jamais être sélectionné ; environ 36,8 % des observations restent donc en dehors de chaque bootstrap et sont dites *out-of-bag* pour le modèle $\hat{f}_i$ considéré.

Pour une observation (x_j, y_j), on note

$$S_j = \{\, i \in \{1, \dots, m\} \mid (x_j, y_j) \notin D_i \,\}$$

l'ensemble des indices de modèles qui n'ont pas vu ce point à l'entraînement. On peut alors définir une prédiction OOB par

$$\tilde{f}_j(x_j) = \frac{1}{|S_j|} \sum_{i \in S_j} \hat{f}_i(x_j)$$

en régression, ou par vote majoritaire restreint aux $i \in S_j$ en classification. Une estimation de l'erreur OOB est donnée par

$$\varepsilon_{\text{OOB}} = \frac{1}{n} \sum_{j=1}^{n} \ell\big(\tilde{f}_j(x_j), y_j\big),$$

où ℓ est la fonction de perte (MSE, entropie croisée, etc.). Cette quantité fournit une approximation fiable de l'erreur de généralisation, sans avoir besoin de réserver un jeu de validation distinct [2, 25, 19].

Réduction de la variance. Le principal intérêt du bagging est de réduire la variance des prédictions. Dans un scénario idéal où les modèles $\hat{f}_i$ seraient indépendants et de variance v, la variance de la moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x)$$

serait v/m. En pratique, les modèles ne sont pas indépendants ; si l'on note ρ la corrélation moyenne entre deux prédicteurs, on obtient approximativement [6] :

$$\mathrm{Var}\big[\hat{f}(x)\big] \approx \rho v + \frac{1-\rho}{m}\, v.$$

Quand m augmente, le second terme décroît comme $1/m$ et la variance tend vers ρv. Le bagging est donc particulièrement pertinent lorsque les modèles de base sont peu biaisés mais très variables (comme les arbres profonds) : il stabilise leurs prédictions sans augmenter significativement le biais.

Exemples d'algorithmes baggés. *Forêt aléatoire.* La forêt aléatoire (*Random Forest*) est l'extension la plus célèbre du bagging. En plus du bootstrapping, elle introduit une randomisation des caractéristiques (*features*) : à chaque division d'un nœud de l'arbre, seul un sous-ensemble aléatoire de variables est considéré. Cela décorrèle les arbres et renforce encore la réduction de variance [2, 19].

Un aperçu schématique de l'algorithme Random Forest est donné par la figure 1.2.

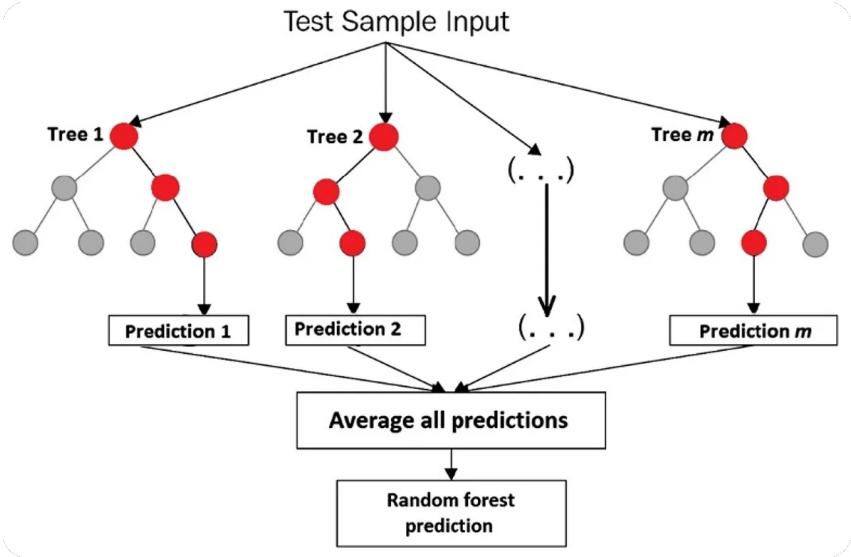

FIGURE 1.2 – Aperçu de l'algorithme Random Forest.

Exemple : bagging avec k plus proches voisins. Le bagging ne se limite pas aux arbres. Par exemple, appliqué au classifieur des k plus proches voisins (k-NN), il permet de stabiliser un modèle naturellement très sensible aux fluctuations locales des données. On génère m jeux bootstrap D_i, chacun servant de base à un modèle k-NN (qui consiste essentiellement à mémoriser D_i). Pour une nouvelle observation x, on agrège les m prédictions k-NN par vote ou moyenne des probabilités estimées [13, 3].

Dans les bibliothèques modernes comme scikit-learn, ces variantes sont disponibles via la classe `BaggingClassifier` en choisissant simplement l'estimateur de base (arbre, k-NN, etc.) et en activant le calcul de la « OOB score » pour obtenir automatiquement une estimation interne de l'erreur de généralisation.

1.3.2 Boosting

Le boosting regroupe une famille de méthodes séquentielles où les modèles sont entraînés les uns après les autres, chacun se concentrant sur les erreurs commises par les précédents. Contrairement au bagging qui agit surtout sur la variance, le boosting cherche principalement à réduire le biais, en construisant progressivement un modèle additif puissant à partir de modèles de base faibles [11, 27, 3, 16].

Principe général. On considère un ensemble d'entraînement $\{(x_i, y_i)\}_{i=1}^n$ avec $y_i \in \{-1, +1\}$ pour fixer les idées. Le boosting maintient une distribution de poids $D_t(i)$ sur les exemples. À chaque itération t :

1. un *apprenant faible* h_t est entraîné sur les données pondérées par D_t ;
2. son erreur pondérée $\varepsilon_t = \sum_i D_t(i)\, \mathbb{I}\big(h_t(x_i) \neq y_i\big)$ est évaluée ;
3. un coefficient α_t mesurant la confiance accordée à h_t est calculé ;
4. les poids D_t sont mis à jour pour donner plus d'importance aux exemples mal prédits.

Le modèle final est une combinaison pondérée des h_t. Sous l'hypothèse de « faible apprentissage » (chaque h_t fait un peu mieux que le hasard), on peut montrer que l'erreur d'entraînement décroît de manière exponentielle avec le nombre d'itérations [11, 27].

AdaBoost : formulation et réduction du biais. AdaBoost (Adaptive Boosting) est l'algorithme de boosting le plus classique [11, 27]. Sa version binaire peut se décrire comme suit.

1. **Initialisation.** On part d'une distribution uniforme

$$D_1(i) = \frac{1}{n}, \quad i = 1, \ldots, n.$$

2. **Pour** $t = 1, \ldots, T$ **:**
   (a) on entraîne un modèle faible $h_t : \mathcal{X} \to \{-1, +1\}$ en minimisant l'erreur pondérée

$$\varepsilon_t = \sum_{i=1}^{n} D_t(i)\, \mathbb{I}\big(h_t(x_i) \neq y_i\big),$$

   avec la contrainte $\varepsilon_t < 1/2$;

   (b) on définit le poids du modèle

$$\alpha_t = \frac{1}{2} \log\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right) ;$$

   (c) on met à jour les poids des exemples :

$$D_{t+1}(i) = \frac{D_t(i)\, \exp\big(-\alpha_t y_i h_t(x_i)\big)}{Z_t},$$

   où Z_t est un facteur de normalisation assurant $\sum_i D_{t+1}(i) = 1$. Les exemples mal classés ($y_i h_t(x_i) < 0$) voient leur poids augmenter, ce qui force h_{t+1} à se focaliser sur eux.

Le classifieur final est

$$H(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

Théoriquement, on peut montrer que l'erreur d'entraînement de H est majorée par [27, 11]

$$\mathrm{err}_{\mathrm{train}}(H) \leq \prod_{t=1}^{T} Z_t \leq \exp\left(-2\sum_{t=1}^{T} \gamma_t^2\right), \quad \text{où } \gamma_t = \tfrac{1}{2} - \varepsilon_t.$$

Si chaque apprenant faible est au moins un peu meilleur que le hasard ($\gamma_t \geq \gamma > 0$), l'erreur d'entraînement décroît donc comme $\exp(-2\gamma^2 T)$. En pratique, cela signifie que le biais du modèle global diminue rapidement avec le nombre d'itérations, tant que les apprenants restent effectivement faibles et diversifiés.

On peut aussi interpréter AdaBoost comme une méthode qui minimise de façon gloutonne la perte exponentielle

$$L(f) = \sum_{i=1}^{n} \exp\bigl(-y_i f(x_i)\bigr), \quad f(x) = \sum_{t=1}^{T} \alpha_t h_t(x),$$

par une descente de gradient coordonnée dans l'espace des fonctions. Cette vision explique pourquoi la marge $y_i f(x_i)$ joue un rôle central dans la performance [27].

En pratique, AdaBoost est souvent implémenté avec des *decision stumps* (arbres de profondeur 1) comme apprenants faibles ; dans scikit-learn, on peut l'utiliser via la classe `AdaBoostClassifier` avec un `DecisionTreeClassifier(max_depth=1)` comme estimateur de base. La complexité est en gros proportionnelle à T fois le coût d'entraînement d'un apprenant faible, et l'algorithme est naturellement itératif.

Gradient boosting : point de vue en espace de fonctions. Le gradient boosting généralise l'idée du boosting à toute fonction de perte différentiable [12]. On cherche à approximer une fonction $F^\star$ en construisant un modèle additif

$$F_T(x) = \sum_{t=1}^{T} \nu\, \beta_t\, h_t(x),$$

où h_t sont des apprenants faibles (souvent des arbres de décision), β_t des coefficients et $\nu \in (0, 1]$ un facteur de « shrinkage » (taux d'apprentissage).

À chaque itération t :

1. on dispose d'un modèle courant F_{t-1} ;
2. on calcule, pour chaque exemple, les *pseudo-résidus* correspondant au gradient négatif de la perte par rapport aux prédictions :

$$r_{it} = -\left.\frac{\partial}{\partial F(x_i)} L\bigl(y_i, F(x_i)\bigr)\right|_{F=F_{t-1}};$$

   pour une perte quadratique, on obtient simplement $r_{it} = y_i - F_{t-1}(x_i)$;
3. on ajuste un apprenant faible h_t pour approximer ces pseudo-résidus, typiquement par

régression aux moindres carrés :

$$h_t \approx \arg\min_h \sum_{i=1}^{n} \big(r_{it} - h(x_i)\big)^2;$$

4. on choisit un pas optimal β_t (par recherche de ligne) :

$$\beta_t = \arg\min_\beta \sum_{i=1}^{n} L\big(y_i, F_{t-1}(x_i) + \beta\, h_t(x_i)\big),$$

puis on met à jour

$$F_t(x) = F_{t-1}(x) + \nu\, \beta_t\, h_t(x).$$

Pour la régression (perte quadratique), cela revient à ajuster successivement les résidus $y_i - F_{t-1}(x_i)$; pour la classification logistique, les pseudo-résidus correspondent à une erreur sur les probabilités estimées. Ainsi, chaque itération corrige systématiquement l'erreur résiduelle, ce qui diminue le biais tout en permettant un contrôle fin de la complexité via ν, la profondeur des arbres et le nombre d'itérations [12].

Des implémentations standards existent dans scikit-learn (`GradientBoostingClassifier`, `GradientBoostingRegressor`) ainsi que dans des bibliothèques dédiées comme XGBoost et LightGBM.

XGBoost et LightGBM : optimisations modernes. XGBoost [4] et LightGBM [21] sont des implémentations très optimisées du gradient boosting sur arbres de décision, qui ajoutent plusieurs idées clés :

— utilisation d'un développement de Taylor d'ordre 2 de la perte pour calculer, pour chaque feuille candidate, le gain associé à une scission en fonction de la somme des gradients G et des hessiennes H ;

— régularisation explicite des arbres via des termes de pénalisation L_2 sur les poids des feuilles et une pénalité sur le nombre de feuilles (paramètres λ et γ) ;

— sous-échantillonnage des observations et des variables (*row/column subsampling*) pour réduire la corrélation entre arbres et accélérer l'entraînement ;

— structures de données basées sur des histogrammes et croissance feuille-par-feuille (LightGBM) pour optimiser l'utilisation de la mémoire et du CPU.

Par exemple, dans XGBoost, le gain d'une scission est approximativement donné par [4]

$$\text{Gain} = \frac{1}{2}\left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right) - \gamma,$$

où G_L, H_L (resp. G_R, H_R) sont les sommes des gradients et hessiennes dans les feuilles gauche et droite. Cette expression permet de choisir efficacement les meilleures scissions.

Du point de vue pratique, XGBoost et LightGBM sont devenus des standards de facto pour les problèmes tabulaires de grande dimension, grâce à leur excellent compromis entre biais (faible) et variance contrôlée par une régularisation riche et de nombreux hyperparamètres.

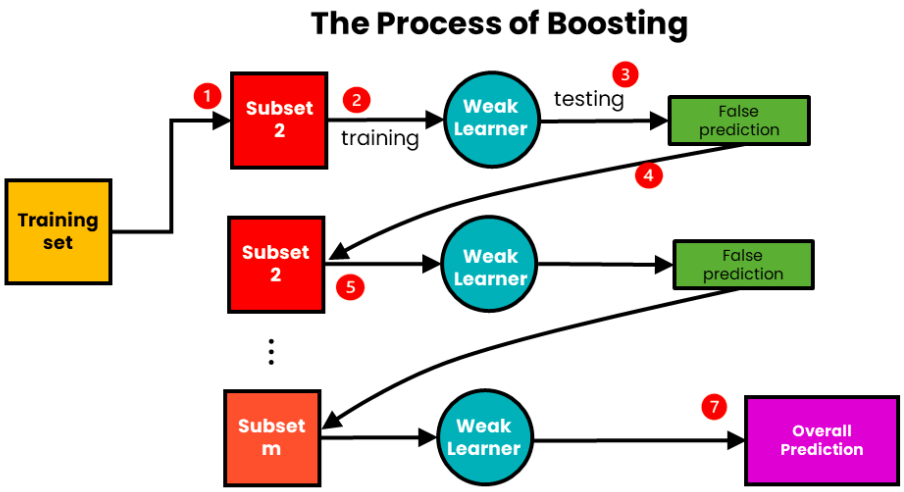

FIGURE 1.3 – Schéma pédagogique du boosting : chaîne de modèles où chaque étape corrige les erreurs de la précédente.

1.3.3 Stacking (stacked generalization)

Le stacking utilise un modèle d'apprentissage pour apprendre comment combiner au mieux les prédictions de plusieurs modèles de base [34, 17].

Architecture en couches.

— **Niveau 0 (base learners)** : des modèles hétérogènes (par exemple SVM, forêt aléatoire, k-NN) sont entraînés sur les données.

— **Méta-caractéristiques** : les prédictions de ces modèles servent de nouvelles variables d'entrée.

— **Niveau 1 (méta-modèle)** : un dernier classifieur (souvent une régression logistique) apprend à pondérer ces prédictions.

Rôle de la validation croisée. Pour éviter le surapprentissage (le méta-modèle pourrait favoriser des modèles qui ont simplement mémorisé le jeu d'entraînement), on utilise des prédictions *out-of-fold*. Chaque modèle de base prédit des données qu'il n'a pas vues durant son entraînement partiel, ce qui garantit que le méta-modèle apprend sur des performances réelles de généralisation.

1.3.4 Voting (vote majoritaire vs vote probabiliste)

Le vote est la méthode de fusion la plus simple pour les modèles parallèles [5, 14].

Hard voting (vote majoritaire). Chaque modèle donne une étiquette de classe. On choisit la classe qui a reçu le plus de suffrages. Par exemple :

— Modèle A → « chien »

— Modèle B → « chat »

— Modèle C → « chien »

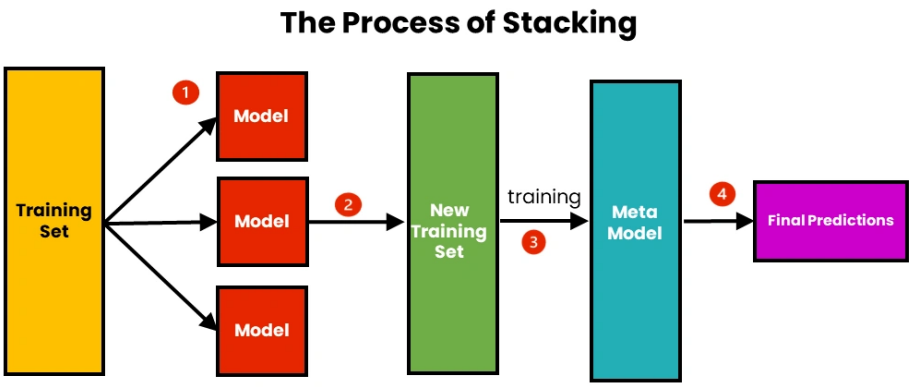

FIGURE 1.4 – Schéma pédagogique du stacking : architecture en couches avec base learners et méta-modèle.

La décision finale est alors « chien » (deux voix contre une).

Soft voting (vote probabiliste). On calcule la moyenne des probabilités prédites pour chaque classe. Cette méthode est plus précise car elle accorde plus d'importance aux modèles « sûrs d'eux ».

Considérons par exemple deux classes (chat, chien) et trois modèles :

— Modèle 1 : probabilités $[P(\text{chien}) = 0{,}4,\ P(\text{chat}) = 0{,}6] \Rightarrow$ prédit « chat »

— Modèle 2 : probabilités $[0{,}4,\ 0{,}6] \Rightarrow$ prédit « chat »

— Modèle 3 : probabilités $[0{,}9,\ 0{,}1] \Rightarrow$ prédit « chien »

En hard voting, la classe « chat » (deux voix) l'emporte. En soft voting, on calcule :

$$\overline{P}(\text{chien}) = \frac{0{,}4 + 0{,}4 + 0{,}9}{3} \approx 0{,}56, \qquad \overline{P}(\text{chat}) = \frac{0{,}6 + 0{,}6 + 0{,}1}{3} \approx 0{,}43.$$

La décision finale est alors « chien » : la forte conviction du modèle 3 compense l'incertitude des deux autres.

1.4 Analyse comparative des approches

Le choix d'une méthode d'ensemble dépend des contraintes du projet (volume de données, temps de réponse, précision requise, robustesse au bruit, coût de calcul, etc.).

Le tableau suivant résume quelques caractéristiques essentielles des trois grandes familles :

TABLE 1.2 – Comparaison synthétique bagging / boosting / stacking

Critère	Bagging	Boosting	Stacking
Objectif principal	Réduire la variance	Réduire le biais	Maximiser la précis
Type de modèles	Homogènes (arbres profonds)	Homogènes (arbres peu profonds)	Hétérogènes (divers
Entraînement	Parallèle (indépendant)	Séquentiel (dépendant)	Parallèle + méta-a
Sensibilité au bruit	Faible (robuste)	Élevée (risque de sur-ajuster)	Variable
Risque d'overfitting	Faible	Élevé (si non régularisé)	Moyen (si données
Coût de calcul	Efficace (parallélisable)	Lent (entraînement en série)	Très élevé

1.5 Aspects architecturaux des systèmes multi-classifieurs

L'architecture d'un système multi-classifieurs définit la manière dont les informations circulent entre les composants et la façon dont les ressources sont exploitées.

1.5.1 Architectures de contrôle de flux

On distingue trois organisations principales :

1. **Architecture parallèle** : tous les classifieurs opèrent indépendamment et simultanément. La fusion intervient à la fin. Cette structure est neutre (aucun classifieur n'influence les autres) et hautement parallélisable sur du matériel multi-cœurs.
2. **Architecture séquentielle (en série)** : les classifieurs sont organisés en cascade. Un premier classifieur rapide effectue une prédiction ; s'il échoue ou exprime une faible confiance, l'exemple est passé à un second modèle plus complexe. Cela permet d'économiser des ressources de calcul en ne sollicitant les modèles coûteux que pour les cas difficiles.
3. **Architecture hybride** : elle combine les deux approches. Par exemple, un premier étage parallèle peut générer un consensus, dont le résultat est validé par un classifieur de contrôle séquentiel.

Ces formes peuvent être illustrées par des figures dédiées aux architectures parallèle et séquentielle.

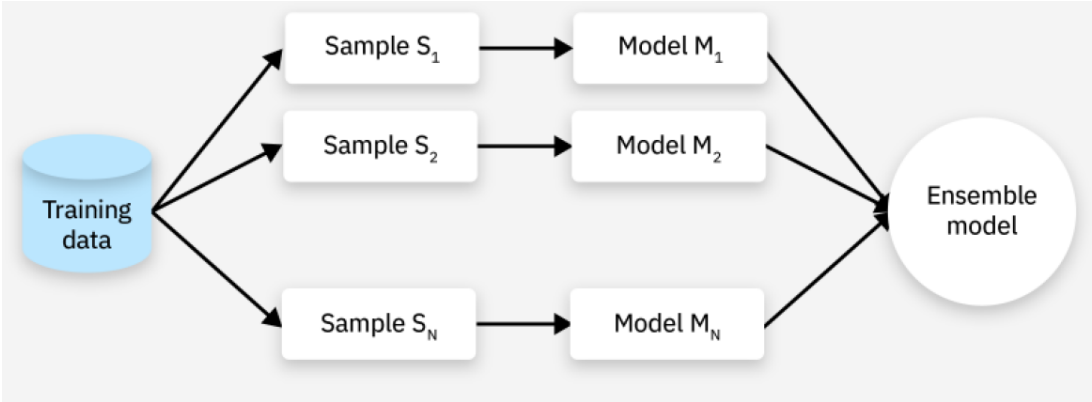

FIGURE 1.5 – Architecture parallèle pour un système multi-classifieurs.

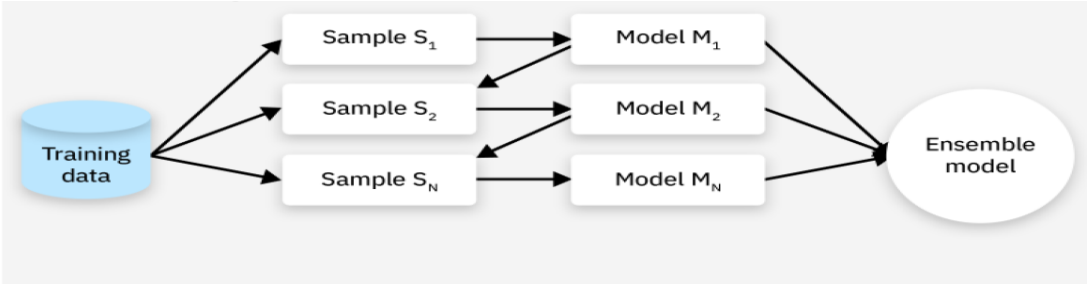

FIGURE 1.6 – Architecture séquentielle pour un système multi-classifieurs.

1.5.2 Systèmes de sélection dynamique (DS)

Une tendance majeure consiste à ne pas utiliser tous les classifieurs pour chaque exemple, mais à sélectionner « à la volée » le ou les modèles les plus compétents [8, 28, 22, 7, 9].

— **Logique de compétence locale** : on définit une « région de compétence » autour d'un exemple test (en utilisant par exemple ses plus proches voisins dans le jeu d'entraînement). On évalue ensuite quels modèles du pool ont été les plus performants dans cette zone spécifique.

— **DCS (dynamic classifier selection)** : on choisit l'unique meilleur modèle pour l'exemple donné.

— **DES (dynamic ensemble selection)** : on construit un sous-ensemble de modèles experts pour cette région particulière.

1.6 Avantages et limites de la combinaison

1.6.1 Avantages

1. **Précision supérieure** : les ensembles dépassent quasi systématiquement les modèles uniques en combinant des perspectives complémentaires.
2. **Généralisation et robustesse** : en lissant les prédictions, les systèmes MCS sont moins sensibles aux anomalies statistiques d'un seul jeu de données.
3. **Traitement des données complexes** : ils permettent de diviser un problème complexe en sous-tâches traitées par des experts dédiés (*divide and conquer*).

1.6.2 Limites

1. **Coût en ressources** : l'entraînement de dizaines ou de centaines de modèles consomme du temps CPU, de la mémoire et de l'énergie.
2. **Opacité (« black box »)** : il est extrêmement difficile d'expliquer pourquoi un ensemble de plusieurs centaines d'arbres de décision a pris une décision spécifique.
3. **Complexité de maintenance** : gérer un pipeline avec plusieurs modèles hétérogènes demande une infrastructure plus lourde qu'un modèle monolithique.

Conclusion

La combinaison de classifieurs représente une approche mature et extrêmement puissante de l'apprentissage automatique. En orchestrant judicieusement la diversité et en s'appuyant sur des fondements statistiques rigoureux, les systèmes multi-classifieurs permettent de transformer une collection de modèles imparfaits en un outil de décision d'une fiabilité exceptionnelle [20, 30, 13, 10].

Chapitre 2

Combinaison de classifieurs et explicabilité

Introduction

Ce chapitre explore la relation entre les méthodes d'ensemble — qui combinent plusieurs modèles pour obtenir de meilleures prédictions — et l'explicabilité en apprentissage automatique. Nous verrons pourquoi ces modèles, bien que très performants, deviennent difficiles à interpréter, comment mesurer et maîtriser leur complexité, quelles stratégies existent pour les expliquer, et comment trouver le bon équilibre entre performance et compréhension. Chaque concept est illustré par un cas concret : le diagnostic précoce de la maladie rénale chronique (CKD).

L'apprentissage automatique moderne est traversé par une tension fondamentale : la quête d'une précision maximale s'oppose souvent à l'exigence de transparence et de compréhensibilité. Au cœur de cette tension se trouvent les **méthodes d'ensemble** (ou *combinaisons de classeurs*), qui agrègent plusieurs modèles pour former un système hautement performant.

Des algorithmes comme les *Random Forests*, le *Gradient Boosting* ou le *Stacking* dominent les compétitions de Machine Learning sur données tabulaires. Cependant, leur puissance s'accompagne d'une **opacité systémique** : plus un modèle d'ensemble est précis, moins il est facile à expliquer. Cette réalité a conduit au développement de l'*eXplainable Artificial Intelligence* (**XAI**), un domaine entier dédié à rendre les décisions algorithmiques compréhensibles.

Notre cas d'étude. Tout au long de ce chapitre, nous utiliserons un exemple concret : le déploiement d'un modèle d'ensemble pour le **diagnostic précoce de la maladie rénale chronique** (CKD — *Chronic Kidney Disease*). Le jeu de données de référence (UCI Machine Learning Repository) contient **400 patients** décrits par **24 variables médicales** : taux d'hémoglobine, pression artérielle, gravité spécifique de l'urine, présence de diabète, etc. La variable cible est binaire : CKD présente ou absente. Ce contexte médical illustre parfaitement l'enjeu : un modèle ultra-précis mais inexplicable est inutilisable par un médecin, car une décision thérapeutique engage la vie d'un patient.

2.1 Le Problème de la Boîte Noire

2.1.1 Qu'est-ce qu'une boîte noire en Machine Learning ?

Définition (Boîte noire (*Black Box*)). Un modèle est dit **boîte noire** lorsque la logique reliant ses entrées (les données du patient, par exemple) à ses sorties (la prédiction) est trop complexe pour être comprise directement par un être humain, même si la prédiction est correcte.

Pour bien saisir ce concept, partons d'une analogie simple. Imaginez deux médecins :

— **Médecin A** dit : "Le taux d'hémoglobine de votre patient est à 9 g/dL, ce qui est bien en dessous du seuil de 12 g/dL. Combiné à une gravité spécifique anormale de 1.010, je diagnostique une insuffisance rénale."

— **Médecin B** dit : "Mon algorithme donne un score de 0.88. Je ne sais pas exactement pourquoi, mais il dit CKD."

Le médecin A raisonne de manière **transparente** : son raisonnement est traçable, vérifiable, contestable. Le médecin B incarne la boîte noire : la prédiction existe, mais la logique est invisible.

En apprentissage automatique, la **transparence** s'évalue selon trois niveaux :

1. **Simulabilité** : un humain peut-il refaire mentalement le calcul du modèle ? Un arbre de décision de 5 nœuds est simulable. Une forêt de 500 arbres ne l'est pas.
2. **Décomposabilité** : chaque paramètre du modèle a-t-il une signification intuitive ? Dans une régression linéaire, le coefficient associé au taux d'hémoglobine est directement interprétable. Dans un ensemble de 300 arbres, ce n'est plus le cas.
3. **Transparence algorithmique** : peut-on prouver mathématiquement comment l'algorithme converge ? La régression logistique a une surface d'erreur convexe avec un minimum global unique : sa convergence est prouvable. Les ensembles profonds n'ont pas cette propriété.

2.1.2 Comment l'agrégation crée-t-elle l'opacité ?

Un arbre de décision isolé et peu profond est parfaitement lisible. Par exemple :

SI hémoglobine < 12 ***ET*** *gravité spécifique* < 1.015 ***ALORS*** *CKD probable*

Ce chemin logique peut être suivi par n'importe quel médecin. Mais dès que l'on agrège **plusieurs centaines** de tels arbres — comme dans une Random Forest ou un Gradient Boosting — la prédiction finale devient :

$$\hat{y} = \sum_{m=1}^{M} h_m(x) \tag{2.1}$$

où h_m est la prédiction du m-ième arbre et M le nombre total d'arbres. Cette somme peut réunir 200, 500, voire 1 000 arbres. Chaque arbre a **ses propres règles**, **ses propres seuils**, et certains arbres peuvent *contredire* d'autres. La frontière de décision globale devient une surface ultra-complexe, non linéaire et multidimensionnelle qu'aucun analyste ne peut visualiser directement.

Attention. L'opacité n'est **pas** due à la complexité d'un arbre individuel, mais à la **dimension combinatoire** des interactions entre tous les arbres. Agréger 500 modèles simples produit quelque chose d'intrinsèquement complexe, même si chaque pièce était compréhensible séparément.

2.1.3 Pourquoi l'explicabilité est-elle indispensable ?

L'opacité des ensembles n'est pas qu'un problème académique : elle a des conséquences **légales**, **éthiques** et **cliniques** directes.

Conséquences légales et réglementaires

Des législations comme le **RGPD** européen (articles 13, 14, 15 et 22) imposent aux organisations de fournir des explications claires lorsqu'une décision automatisée affecte un individu. En médecine, les normes de la FDA américaine ou les certifications CE européennes exigent la traçabilité des systèmes d'aide au diagnostic. Un hôpital qui déploie un modèle CKD sans pouvoir expliquer ses décisions s'expose à des sanctions légales sévères.

Risques cliniques spécifiques

Dans notre cas fil rouge, deux phénomènes dangereux peuvent survenir en l'absence d'explicabilité :

— **Biais d'automatisation** : le médecin fait aveuglément confiance au score de 0.88 sans le remettre en question, même quand les biomarqueurs affichés ne semblent pas alarmants.

— **Fatigue des alarmes** : à l'inverse, le médecin ignore systématiquement les alertes car il ne comprend pas leur logique, laissant passer des cas urgents.

Risque de corrélations fallacieuses

Les modèles d'ensemble peuvent apprendre des **corrélations parasites** plutôt que des relations causales véritables. Sur le jeu de données CKD, si un biais de recrutement a systématiquement associé un biomarqueur démographique (comme l'âge ou la région) à la maladie, le modèle exploitera cette corrélation sans que le praticien ne puisse le détecter — sauf s'il dispose d'un outil d'explicabilité.

Le mur de l'adoption clinique. Un nephrologue expérimenté refuse légitimement de modifier le traitement d'un patient uniquement sur la base d'une équation opaque $\hat{y} = \sum_{m=1}^{M} h_m(x)$. Il doit pouvoir vérifier que le modèle s'appuie sur des biomarqueurs physiologiquement pertinents (hémoglobine, créatinine, gravité spécifique) et non sur des artefacts du jeu de données. C'est précisément le rôle de l'XAI : déverrouiller la boîte noire pour instaurer une confiance scientifique.

2.2 Complexité des Ensembles de Modèles

Comprendre pourquoi les ensembles sont opaques nécessite d'examiner leur **complexité** sous trois angles : leur structure interne, leur capacité à mémoriser les données, et la dynamique algorithmique qui les construit.

2.2.1 La dimension VC : mesurer la capacité d'un modèle

Définition (Dimension de Vapnik-Chervonenkis (Dimension VC)). La **dimension VC** mesure la richesse d'un modèle : c'est le nombre maximal de points que le modèle peut séparer de toutes les façons possibles, quelle que soit leur configuration. Plus la dimension VC est grande, plus le modèle est flexible — et plus il risque de mémoriser le bruit plutôt que d'apprendre des règles générales.

Exemple concret. Un arbre de décision de profondeur h peut capturer des interactions entre h variables simultanément. Un arbre de profondeur 3 peut relier : "hémoglobine ET âge ET pression artérielle'. Quand on agrège M tels arbres dans une Random Forest, la dimension VC de l'ensemble croît **exponentiellement** : l'espace des décisions possibles explose, et il devient impossible pour un analyste de raisonner sur toutes ces interactions imbriquées.

Comment limiter cette explosion ?

Un hyperparamètre clé : `max_leaf_nodes = k`. En limitant chaque arbre à k feuilles terminales, on plafonne les interactions capturables à l'ordre $k-1$. Cela a un double bénéfice :

- **Réduction du surapprentissage** : le modèle n'apprend pas les particularités des 400 patients du jeu CKD au lieu des règles physiologiques.
- **Meilleure explicabilité** : moins d'interactions profondes signifie des contributions de variables plus stables et plus facilement isolables.

2.2.2 La borne de généralisation : pourquoi la complexité nuit à l'explication

La théorie de l'apprentissage statistique nous donne une garantie mathématique importante : l'erreur réelle d'un modèle (sur des données qu'il n'a jamais vues) est bornée par son erreur d'entraînement **plus** un terme pénalisant sa complexité. Dit simplement :

$$\text{Erreur réelle} \leq \text{Erreur d'entraînement} + \underbrace{2 \times \text{Complexité du modèle}}_{\text{pénalité}} + \text{terme statistique} \tag{2.2}$$

Implication pratique. Si l'on ajoute aveuglément des arbres très profonds dans un ensemble XGBoost entraîné sur 400 patients, la pénalité de complexité augmente : le modèle surapprentit les données d'entraînement et ses performances réelles se dégradent. De plus, un modèle qui surapprentit interprète le *bruit* des données comme un signal réel : ses explications deviennent non seulement inexactes, mais **physiologiquement absurdes**.

Point clé. La complexité est l'ennemi à la fois de la **généralisation** et de l'**explicabilité**. Maîtriser la complexité d'un ensemble, c'est simultanément améliorer ses prédictions sur de nouveaux patients *et* rendre ses décisions plus compréhensibles.

2.2.3 Bagging et Boosting : deux architectures, deux types de complexité

Les deux grandes familles d'ensembles abordent la complexité de manières fondamentalement différentes. Le tableau 2.1 synthétise leurs différences.

TABLE 2.1 – Comparaison algorithmique : Bagging vs Boosting

Caractéristique	**Bagging** (ex. : Random Forest)	**Boosting** (ex. : XGBoost)
Construction	Parallèle et indépendante	Séquentielle et dépendante
Objectif	Réduire la **variance**	Réduire le **biais**
Surapprentissage	Stable, moins sensible	Sensible, nécessite une régularisation stricte
Coût de calcul	Linéaire, parallélisable	Séquentiel ($\approx$14$\times$ plus lent)
Diversité	Par tirage aléatoire (bootstrap)	Par correction ciblée des erreurs
Risque d'opacité	Modéré (votes indépendants)	Élevé (corrections cumulées et dépendantes)

Le Bagging en détail

Le Bagging (Bootstrap Aggregating) construit chaque arbre sur un **sous-échantillon aléatoire** des données d'entraînement. Pour la Random Forest, une double stochasticité est introduite : tirage aléatoire des patients *et* sélection aléatoire des variables à chaque nœud. L'agrégation finale se fait par **vote majoritaire** (classification) ou **moyenne** (régression).

Exemple (Bagging sur le jeu CKD). Imaginez 3 médecins (les 3 arbres) qui examinent chacun un sous-ensemble différent des 400 dossiers patients :

— Médecin 1 : basé sur l'hémoglobine et la pression artérielle $\rightarrow$ **CKD**

— Médecin 2 : basé sur la créatinine et l'albumine $\rightarrow$ **CKD**

— Médecin 3 : basé sur l'âge et le glucose $\rightarrow$ **sain**

Vote final (2 contre 1) : **CKD**. La prédiction est stable, mais il est difficile de savoir quel médecin a eu le plus d'influence.

Le Boosting en détail

Le Boosting construit les arbres **séquentiellement** : chaque nouvel arbre se concentre sur les cas que les arbres précédents ont mal classés. Pour le Gradient Boosting (GBRT), cela revient à ce que chaque arbre h_m approxime le **gradient de l'erreur** du modèle précédent. Le modèle final est :

$$F_M(x) \;=\; F_{M-1}(x) \;+\; h_M(x) \tag{2.3}$$

Le Boosting réduit fortement le biais (il détecte des signaux subtils que le Bagging rate), mais requiert un contrôle minutieux via des hyperparamètres comme le taux d'apprentissage (`learning_rate`) et les pénalités L_1/L_2 pour éviter l'explosion de la variance.

Gestion de la complexité sur les données CKD. Avec seulement 400 patients et 24 variables, le risque de surapprentissage est élevé. Un XGBoost mal configuré mémoriserait les particularités des 400 cas plutôt que les règles physiologiques de la CKD. La stratégie recommandée :

1. Limiter `max_depth` à 3 ou 4 (interactions d'ordre faible).
2. Fixer `max_leaf_nodes` pour brider l'expressivité de chaque arbre.
3. Utiliser un `learning_rate` faible (0.05–0.1) pour forcer une correction progressive.

Ces contraintes ont un double effet positif : elles préviennent le surapprentissage *et* rendent les contributions des biomarqueurs clés (hémoglobine, gravité spécifique) plus stables et plus isolables par les méthodes XAI.

2.3 Stratégies d'Explicabilité des Ensembles (XAI)

Puisque la nature agrégée des ensembles génère une opacité inévitable, la communauté scientifique a développé trois grandes familles de stratégies pour en extraire du sens. Nous les présentons ici avec leurs fondements, leurs forces et leurs limites, illustrées sur le cas CKD.

2.3.1 Les valeurs SHAP : une explication fondée sur la théorie des jeux

Définition (SHAP — SHapley Additive exPlanations). **SHAP** est une méthode qui attribue à chaque variable d'entrée une **valeur d'importance** (ϕ_i) pour une prédiction donnée. Cette valeur représente la contribution marginale de la variable, en moyenne sur toutes les combinaisons possibles de variables. Le fondement est la **théorie des jeux coopératifs** de Lloyd Shapley : les variables sont les "joueurs' d'une coalition cherchant à "gagner' (expliquer la prédiction).

L'idée intuitive : le jeu de coalition

Voici l'intuition derrière SHAP avec un exemple du jeu CKD. Imaginons que le modèle XGBoost prédit **88 % de risque CKD** pour un patient donné. On cherche à répondre : "Quelle variable a le plus contribué à ce score ?'

La valeur SHAP d'une variable i s'obtient en posant la question : "En moyenne, quelle est la **différence de prédiction** quand on ajoute la variable i à une coalition de variables, par rapport à la coalition sans i ?'

Formellement :

$$\phi_i \;=\; \sum_{S \subseteq \text{toutes variables} \setminus \{i\}} \frac{|S|!\,(|M|-|S|-1)!}{|M|!}\,\big[f(S \cup \{i\}) - f(S)\big] \tag{2.4}$$

Pas de panique avec cette formule : il faut retenir que le terme factoriel est un **poids combinatoire** qui garantit l'équité de la répartition, et que $f(S \cup \{i\}) - f(S)$ mesure l'apport marginal de la variable i.

Les trois propriétés garanties par SHAP

SHAP est la **seule** méthode d'attribution qui garantit simultanément :

1. **Précision locale** : la somme de toutes les valeurs SHAP est exactement égale à l'écart entre la prédiction du patient et la prédiction moyenne du modèle :

$$\hat{f}(x) \;=\; \mathbb{E}[\hat{f}] \;+\; \sum_{j=1}^{M} \phi_j \tag{2.5}$$

2. **Missingness** : si une variable n'a aucun impact sur la prédiction, sa valeur SHAP est strictement zéro.
3. **Cohérence** : si une variable devient plus influente dans un modèle modifié, sa valeur SHAP ne peut pas diminuer.

TreeSHAP : rendre SHAP rapide pour les ensembles d'arbres

Le calcul exact des valeurs SHAP nécessite d'évaluer le modèle pour toutes les combinaisons possibles de variables : une complexité $\mathcal{O}(2^M)$, soit $2^{24} \approx 16$ millions d'évaluations pour nos 24 variables CKD. C'est impossible en temps réel.

TreeSHAP résout ce problème en exploitant la structure interne des arbres de décision. Il parcourt les chemins de l'arbre une seule fois et calcule les valeurs SHAP exactes en temps **polynomial** :

$$\mathcal{O}(2^M) \;\longrightarrow\; \mathcal{O}(T \cdot L \cdot D^2) \tag{2.6}$$

où T = nombre d'arbres, L = feuilles par arbre, D = profondeur maximale. Sur un XGBoost entraîné pour la CKD, cela réduit le temps de calcul de *plusieurs heures* à *quelques millisecondes*.

TreeSHAP appliqué au diagnostic CKD. Un nouveau patient est admis avec un score de risque de 88 %. Le médecin lance le module TreeSHAP, qui retourne en quelques millisecondes :

Variable	ϕ_i (valeur SHAP)	Interprétation
Taux d'hémoglobine (faible)	+0.42	Pousse fortement vers CKD
Gravité spécifique urine (basse)	+0.31	Pousse fortement vers CKD
Présence de diabète mellitus	+0.15	Pousse vers CKD
Pression artérielle (normale)	−0.08	Réduit légèrement le risque
Âge du patient (50 ans)	+0.03	Impact marginal
Score moyen du modèle	+0.05	Valeur de référence

Somme des ϕ_i + score moyen = 0.88 (88 %)

Le médecin voit immédiatement que c'est principalement le **faible taux d'hémoglobine** couplé à une **gravité spécifique anormale** qui drive la prédiction — des marqueurs physiologiquement cohérents avec la CKD. La boîte noire est déverrouillée.

2.3.2 LIME : une explication locale par approximation linéaire

Définition (LIME — Local Interpretable Model-agnostic Explanations). **LIME** explique une prédiction individuelle en construisant un **modèle simple** (souvent une régression linéaire) qui imite localement le comportement de la boîte noire au voisinage du point à expliquer. L'idée géométrique : même si la frontière de décision globale est complexe, elle est **localement linéaire** quand on zoome suffisamment.

Fonctionnement en 4 étapes

1. **Perturbation** : on génère des copies artificielles du patient en modifiant légèrement ses variables (bruit gaussien, masquage de valeurs).
2. **Interrogation** : on soumet ces copies à la boîte noire XGBoost pour obtenir leurs prédictions.
3. **Pondération** : les copies les plus proches du patient original reçoivent un poids plus élevé. Le noyau de pondération est exponentiel :

$$\pi_x(z) = \exp\left(-\frac{D(x,z)^2}{\sigma^2}\right) \tag{2.7}$$

   où $D(x, z)$ est la distance entre la copie z et le patient original x.
4. **Ajustement** : on entraîne une régression linéaire sur ces copies pondérées. Les coefficients de cette régression deviennent les "explications' du modèle pour ce patient.

Forces et limites de LIME

Attention. Dans un contexte médical, l'**instabilité de LIME** est un problème critique. Si deux applications successives du même modèle sur le même patient donnent des importances de variables contradictoires, le médecin ne peut pas faire confiance à l'explication. C'est pourquoi SHAP (et en particulier TreeSHAP) est préféré dans les contextes à hauts enjeux.

Table 2.2 – Forces et limites de LIME pour les ensembles

Forces	**Limites**
Rapide à calculer	Instable : deux exécutions sur le même patient peuvent donner des résultats différents
Agnostique : fonctionne avec n'importe quel modèle	La taille du voisinage (σ) est un choix arbitraire
Facile à comprendre visuellement	Échoue sur des frontières très non linéaires
	Pas de garantie mathématique d'équité

2.3.3 Extraction de règles globales : de la forêt aux règles médicales (SIRUS)

TreeSHAP et LIME fournissent des explications **locales** (par patient). Mais comment obtenir une vue **globale** du comportement d'un ensemble ? L'idée est de distiller la forêt en un petit ensemble de **règles simples**.

Définition (Règle de décision). Une règle de décision est une condition logique de la forme :

"**SI** [condition(s)] **ALORS** [prédiction]"

Les règles sont compréhensibles par tout professionnel sans formation en ML.

L'algorithme **SIRUS** (*Stable and Interpretable RUle Set*) aborde ce problème en :

1. Extrayant toutes les règles latentes des arbres de la Random Forest.
2. Filtrant les règles instables (celles qui disparaissent si l'on modifie légèrement les données d'entraînement).
3. Ne conservant qu'un petit nombre de règles stables, maximisant leur stabilité via un indice de Dice-Sørensen.

SIRUS appliqué au diagnostic CKD. L'hôpital souhaite un **protocole de tri rapide** aux urgences, sans serveur de calcul. L'algorithme SIRUS extrait 4 règles cliniques fondamentales de la Random Forest entraînée sur les 400 dossiers CKD :

1. **SI** hémoglobine $< 12.0\,\mathrm{g/dL}$ **ET** gravité spécifique $< 1.015 \Rightarrow$ **Probabilité CKD élevée**
2. **SI** créatinine sérique $> 1.2\,\mathrm{mg/dL}$ **ET** albumine anormale $\Rightarrow$ **Probabilité CKD élevée**
3. **SI** hémoglobine $\geq 12.0\,\mathrm{g/dL}$ **ET** pression normale $\Rightarrow$ **Probabilité CKD faible**
4. **SINON** $\Rightarrow$ **Examen complémentaire requis**

Ces 4 règles, dérivées mathématiquement de millions de nœuds de la forêt, sont mémorisables et intégrables dans un manuel médical. Elles ne couvrent pas tous les cas (d'où la règle 4), mais capturent les situations les plus fréquentes avec une précision validée.

2.3.4 Distillation de connaissances : enseigner la transparence

Définition (Distillation de connaissances (*Knowledge Distillation*)). La **distillation** consiste à entraîner un modèle simple et interprétable ("l'élève') à imiter les prédictions d'un ensemble complexe ("le professeur'). L'élève n'apprend pas les étiquettes binaires brutes (0 ou 1), mais les **probabilités continues** émises par le professeur (par exemple, 0.73 plutôt que "CKD'). Ces probabilités portent plus d'information : elles reflètent l'incertitude du modèle.

Pourquoi les probabilités molles (*soft labels*) sont-elles précieuses ?

Quand l'ensemble XGBoost prédit 0.73 de probabilité CKD (plutôt que bêtement "1'), il communique implicitement : "Ce cas ressemble à 73 % aux profils CKD que j'ai vus, mais il y a 27 % de chance que je me trompe.' Ces nuances probabilistes — appelées **connaissances sombres** (*dark knowledge*) — contiennent des informations sur les frontières de décision et les cas ambigus. L'élève, entraîné sur ces probabilités, acquiert implicitement cette sagesse.

Distillation pour le déploiement mobile CKD (Edge AI). Des médecins itinérants dans des zones rurales disposent de simples tablettes sans connexion cloud. Déployer un XGBoost de 300 arbres est impossible (ressources insuffisantes). La solution :

1. **Professeur** : l'XGBoost est entraîné sur les 400 cas CKD en laboratoire. Il génère des probabilités molles pour chaque patient.
2. **Élève** : un arbre CART de profondeur 4 (quelques kilo-octets, visualisable en entier) est entraîné à reproduire ces probabilités molles.
3. **Résultat** : l'élève est embarqué sur la tablette. Il est transparent (le médecin peut inspecter visuellement chaque nœud de l'arbre), rapide, et bien plus précis qu'un arbre entraîné directement sur les données brutes, car il bénéficie de la sagesse de l'ensemble professeur.

Table 2.3 – Synthèse des stratégies d'explicabilité pour les ensembles

Stratégie	**Portée**	**Avantage principal**	**Limite**
TreeSHAP	Locale + Globale	Fondements mathématiques stricts, exact pour les arbres	Lent sur très grands ensembles
LIME	Locale	Rapide, agnostique	Instable, approximatif
SIRUS	Globale	Règles lisibles par des humains	Perte de nuances non linéaires
Distillation	Globale	Modèle transparent autonome	Peut manquer la précision du professeur

2.4 Compromis Performance–Compréhension

La conception d'un modèle d'ensemble est un exercice d'équilibre : maximiser la précision prédictive **et** maximiser la compréhensibilité sont deux objectifs qui peuvent entrer en conflit. Ce compromis, longtemps perçu comme inévitable, est aujourd'hui beaucoup mieux compris — et partiellement dépassé.

2.4.1 Le front de Pareto : visualiser le compromis

Définition (Optimisation multi-objectif et front de Pareto). Lorsqu'on cherche simultanément à minimiser deux objectifs antagonistes (ici : erreur de prédiction *et* opacité), on ne trouve pas *un* modèle optimal mais une **courbe de solutions optimales** : le **front de Pareto**. Sur ce front, améliorer un objectif dégrade nécessairement l'autre.

Concrètement : chaque point du front de Pareto correspond à un ensemble entraîné avec un certain réglage de ses hyperparamètres. On peut parcourir ce front en ajustant des paramètres tels que la profondeur maximale des arbres, le nombre de variables autorisées, ou des contraintes de monotonie sur certaines variables.

L'objectif mathématique de cette optimisation est de minimiser conjointement plusieurs critères :

$$\min_{\lambda} \Big(\underbrace{\text{Erreur de généralisation}}_{\text{performance}}, \underbrace{N_F}_{\text{nb de variables}}, \underbrace{N_I}_{\text{nb d'interactions}}, \underbrace{N_{NM}}_{\text{effets non monotones}} \Big) \tag{2.8}$$

Les trois derniers termes mesurent l'interprétabilité :

- N_F (*parcimonie des variables*) : un modèle n'utilisant que 3 variables est plus compréhensible qu'un modèle en utilisant 20.
- N_I (*parcimonie des interactions*) : moins les variables interagissent ensemble, plus le modèle se comporte comme une somme de contributions indépendantes — ce qui est facile à expliquer.
- N_{NM} (*effets non monotones*) : les humains appréhendent mieux les relations monotones. On peut par exemple forcer le modèle à respecter : "plus la pression artérielle est élevée, plus le risque rénal est grand'.

2.4.2 Quantifier l'interprétabilité : les métriques de fidélité et stabilité

Pour comparer objectivement des méthodes d'explicabilité, il faut des métriques précises. Les deux plus importantes sont :

Fidélité (*Fidelity*)

La fidélité mesure si l'explication reflète vraiment le comportement interne du modèle. On l'évalue par "suppression progressive des variables' (*pixel flipping* ou *AUC deletion*) :

1. On supprime les variables dans l'ordre décroissant de leur importance SHAP (les plus importantes en premier).
2. Si SHAP est fidèle, la performance du modèle doit **chuter brutalement** dès qu'on retire les premières variables.

Une fidélité élevée confirme que SHAP identifie bien les variables réellement utilisées par la boîte noire.

Stabilité (*Robustness*)

La stabilité mesure si l'explication reste cohérente face à de légères variations des données. On ajoute un bruit infime aux données du patient et on vérifie que le classement des variables importantes ne change pas radicalement.

TABLE 2.4 – Stabilité de SHAP vs LIME face au bruit (coefficient de Jaccard)

Jeu de données	**Top-k vars**	**Stabilité SHAP**	**Stabilité LIME**
Iris (faible dim.)	$k = 2$	0.950 (très bon)	1.000 (parfait)
Diabetes (haute dim.)	$k = 3$	0.667 (fragile)	1.000 (apparent)

Note : La stabilité LIME de 1.000 sur Diabetes peut être un artefact de son ajustement linéaire trop rigide.

Attention. L'illusion de transparence. Une étude marquante a démontré qu'un data scientist malveillant peut concevoir un modèle d'ensemble discriminatoire (par exemple, biaisé selon l'origine ethnique) tout en forçant LIME et SHAP à afficher des explications parfaitement équitables en apparence. Les méthodes post-hoc ne sont donc pas infaillibles : dans des contextes adversariaux, elles peuvent générer un *paravent* de transparence cachant un biais réel.

2.4.3 Les modèles hybrides : fusionner performance et transparence

Une approche élégante pour naviguer sur le front de Pareto consiste à concevoir un modèle dont la prédiction est explicitement décomposée en deux parties :

$$h(x) = \underbrace{f^*(x)}_{\text{partie transparente}} + \underbrace{g^*(x)}_{\text{partie boîte noire}} \tag{2.9}$$

— $f^*(x)$: une fonction simple et lisible (régression linéaire, quelques règles) qui capture les relations les plus importantes et les plus claires.

— $g^*(x)$: une fonction complexe (résidu d'une forêt, réseau de neurones) qui absorbe les non-linéarités résiduelles pour booster la précision.

L'idée est de **forcer** la majeure partie du signal explicatif dans f^* et de cantonner g^* à un rôle de correction mineure. On y parvient en pénalisant plus fortement la complexité de g^* que celle de f^* lors de l'entraînement.

2.4.4 Les EBM : des ensembles intrinsèquement transparents

L'**Explainable Boosting Machine** (EBM), développée par Microsoft Research, représente l'état de l'art en matière de modèles *Glass-Box* : des modèles intrinsèquement interprétables mais aussi performants que les boîtes noires.

L'équation d'un EBM est :

$$g(\mathbb{E}[y]) \;=\; \beta_0 \;+\; \sum_{j=1}^{n} f_j(x_j) \;+\; \sum_{i \neq j} f_{ij}(x_i, x_j) \tag{2.10}$$

où :

— $f_j(x_j)$ est une fonction lisse apprise pour la variable j **seule** : on peut la visualiser comme une courbe 2D montrant l'effet de cette variable sur la prédiction.

— $f_{ij}(x_i, x_j)$ capture l'interaction **entre deux variables** : une carte de chaleur 3D.

La clé : chaque f_j est apprise par un Gradient Boosting cyclique sur une seule variable à la fois, avec des arbres minuscules (*stumps*). Cela garantit que la contribution de chaque variable reste **isolable et visualisable**.

TABLE 2.5 – Performance comparée (AUROC) des modèles selon leur niveau de transparence

Domaine	**Régression logistique**	**XGBoost (boîte noire)**	**EBM (Glass-Box)**
Revenus (Adult)	0.907	0.927	**0.928**
Cardio (Heart Disease)	0.895	0.851	**0.898**
Cancer (Breast)	0.995	0.992	**0.995**
Fraude (Credit Fraud)	0.979	0.981	**0.981**

Source : Microsoft Research, InterpretML. L'EBM égale ou dépasse XGBoost tout en restant transparente.

Point clé. Ces résultats empiriques réfutent le **mythe du compromis absolu**. Sur des données structurées (tableaux), il n'y a pas nécessairement à choisir entre performance et transparence. Les EBM atteignent la précision de XGBoost tout en restant totalement explicables : chaque variable a une contribution visualisable et auditée.

2.4.5 Cas fil rouge : atteindre l'optimum de Pareto pour le déploiement CKD

De la boîte noire au modèle cliniquement déployable.

Situation initiale. L'équipe entraîne un XGBoost sur les 24 biomarqueurs CKD. Le modèle atteint une précision de **99.2 %** (AUC $\approx$ 1.0), mais échoue aux tests de parcimonie : il utilise des interactions si profondes que le corps médical le rejette, craignant des artefacts non causaux.

Application de l'optimisation de Pareto. L'équipe intègre des pénalités sévères sur N_F (nombre de variables) et N_{NM} (monotonie forcée pour la pression artérielle et l'albumine). L'al-

gorithme évolutif parcourt l'espace des hyperparamètres et écarte tout ensemble utilisant plus de 5 variables.

Résultat sur le front de Pareto. Le système isole un XGBoost contraint qui ne s'appuie que sur **3 variables** :

1. Le taux d'hémoglobine
2. La gravité spécifique de l'urine
3. La présence d'hypertension

Ce modèle atteint une précision de **99.2 %** (écart-type de 0.8 % sur validation croisée à 5 plis) : identique à la version utilisant les 24 variables. Les bénéfices sont considérables :

— **Coûts réduits** : seulement 3 analyses à réaliser au lieu de 24. Accessible aux zones médicales sous-dotées.

— **Confiance clinique** : les 3 variables sont des marqueurs CKD reconnus. Les explications TreeSHAP deviennent claires et incontestables.

— **Conformité légale** : un modèle mathématiquement audité qui n'utilise aucune variable fallacieuse est juridiquement défendable face au RGPD et aux normes FDA.

Modèle	**Variables**	**Précision**	**Transparence**
XGBoost (24 vars)	24	99.2 %	Nulle (boîte noire)
XGBoost-SHAP post-hoc	24	99.2 %	Partielle (instable)
XGBoost-Pareto (3 vars)	**3**	**99.2 %**	**Élevée (stable)**
EBM (3 vars distillées)	3	98.9 %	**Totale (Glass-Box)**

Ce résultat illustre le message central de ce chapitre : intégrer l'explicabilité dès la conception du modèle n'est pas un frein à la performance, mais le **catalyseur** permettant au Machine Learning de franchir le fossé entre la recherche en laboratoire et le déploiement clinique réel.

Conclusion

Ce chapitre a exploré la tension entre performance et explicabilité dans les méthodes d'ensemble. Voici les points à retenir :

1. **La boîte noire est un problème réel.** Agréger des modèles simples crée une opacité dont les conséquences sont légales (RGPD), éthiques (biais cachés) et cliniques (rejet par les praticiens).
2. **La complexité se mesure.** La dimension VC, les bornes de généralisation et les métriques de parcimonie permettent de quantifier et de contrôler la complexité d'un ensemble. Maîtriser la complexité améliore à la fois la généralisation et l'explicabilité.
3. **Trois familles de stratégies XAI existent.** TreeSHAP (exact, fondé sur la théorie des jeux), LIME (rapide, approximatif), extraction de règles (globale, lisible par les humains) et distillation (modèle transparent autonome).

4. **Le compromis n'est pas inévitable.** Sur des données structurées, les modèles Glass-Box (EBM) et les ensembles optimisés par Pareto peuvent atteindre la précision des boîtes noires tout en restant parfaitement explicables. L'explicabilité n'est pas un frein : c'est une condition de déploiement responsable.

Conclusion générale

Ce rapport a exploré deux piliers de l'apprentissage automatique contemporain : la **combinaison de classifieurs** et l'**explicabilité des modèles**, ainsi que leurs interactions. Plusieurs enseignements en découlent.

La combinaison de classifieurs (bagging, boosting, stacking, voting) permet d'améliorer la précision et la robustesse des prédictions en s'appuyant sur la diversité et l'agrégation des modèles. Les motivations théoriques (réduction de la variance, du biais ou des deux) et les choix architecturaux (génération de l'ensemble, sélection, fusion) conditionnent directement les performances. En contrepartie, ces systèmes deviennent plus complexes : plusieurs apprenants, mécanismes de pondération ou de méta-apprentissage, et donc une décision plus difficile à décomposer et à justifier.

L'explicabilité apparaît comme une exigence complémentaire à la performance. La distinction entre interprétabilité (structure du modèle lisible) et explicabilité (capacité à rendre compte des décisions) permet de clarifier les objectifs. Les méthodes post-hoc (importance des variables, LIME, SHAP, PDP/ICE) offrent des leviers pour expliquer des modèles a priori opaques, en local ou en global. Leur utilisation dans les systèmes décisionnels est renforcée par les enjeux éthiques et réglementaires (transparence, non-discrimination, droit à l'information).

À l'intersection des deux thèmes, le rapport a mis en évidence le **problème de la boîte noire** propre aux ensembles : une décision collective est plus difficile à attribuer à des facteurs précis qu'une décision d'un modèle unique. Les stratégies d'explicabilité des ensembles (explication au niveau de l'agrégation, au niveau des apprenants de base, ou par approximation) permettent de progresser, mais un **compromis entre performance et compréhension** reste inévitable. Selon le contexte (réglementation, domaine critique, acceptabilité utilisateur), le curseur entre complexité du modèle et niveau d'explicabilité requis devra être ajusté.

En synthèse, la combinaison de classifieurs reste un outil puissant pour la qualité prédictive ; l'explicabilité en est le corollaire indispensable dès que les décisions ont un impact humain ou juridique. Les travaux futurs pourront approfondir l'intégration de contraintes d'explicabilité dès la conception des systèmes d'ensemble et l'évaluation standardisée des méthodes d'explication dans ce cadre.

Bibliographie

[1] Analytics Vidhya. Bagging, boosting and stacking : Ensemble learning in ml models. https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/, 2023. Consulté le 23 février 2026.

[2] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2) :123–140, 1996. Article fondateur du bagging.

[3] Jason Brownlee. Bagging vs boosting vs stacking : Which ensemble method wins in 2025 ? https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/, 2025. Consulté le 23 février 2026.

[4] Tianqi Chen and Carlos Guestrin. Xgboost : A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[5] Codefinity. Learn voting : Hard and soft. https://codefinity.com/courses/v2/3172b303-2b09-492b-8bb9-5bbd1bdcef09/81f3a01b-d537-4478-bc23-3ea1895e4f1f/d66aa7bd-877e-4808-ad40-aa051eabbc4a, 2024. Consulté le 23 février 2026.

[6] William W. Cohen. Ensemble methods for machine learning. https://www.cs.cmu.edu/~wcohen/10-601/ensembles.pdf, 2010. Support de cours, Carnegie Mellon University. Consulté le 23 février 2026.

[7] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Dynamic ensemble selection and instantaneous pruning for regression. In *ESANN 2014*, 2014. Consulté le 23 février 2026.

[8] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Deslib : A dynamic ensemble selection library in python. *Journal of Machine Learning Research*, 21(8) :1–5, 2020. Consulté le 23 février 2026.

[9] DESlib project. Welcome to deslib documentation ! https://deslib.readthedocs.io/, 2020. Consulté le 23 février 2026.

[10] Luis Fred. Multiple classifier systems — a brief introduction. https://medium.com/luisfredgs/multiple-classifier-systems-a-brief-introduction-71238d9c794f, 2023. Consulté le 23 février 2026.

[11] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1) :119–139, 1997.

[12] Jerome H. Friedman. Greedy function approximation : A gradient boosting machine. *Annals of Statistics*, 29(5) :1189–1232, 2001.

[13] GeeksforGeeks. A comprehensive guide to ensemble learning. https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/, 2023. Consulté le 23 février 2026.

[14] GeeksforGeeks. Voting classifier. https://www.geeksforgeeks.org/machine-learning/voting-classifier/, 2023. Consulté le 23 février 2026.

[15] Vibhav Gogate. Bias/variance tradeoff and ensemble methods. https://personal.utdallas.edu/~vibhav.gogate/ml/2020f/lectures/EnsembleMethods.pdf, 2020. Support de cours, Université du Texas à Dallas. Consulté le 23 février 2026.

[16] Great Learning. Understanding the ensemble method bagging and boosting. https://www.mygreatlearning.com/blog/bagging-boosting/, 2023. Consulté le 23 février 2026.

[17] Henry Henry. Théorie – apprentissages d'ensemble : bagging, boosting et stacking. https://medium.com/@henryhenry_3807/th%C3%A9orie-apprentissages-densemble-bagging-boosting-et-stacking-f3c8d2273edf, 2023. Consulté le 23 février 2026.

[18] Laurent Heutte. Combinaison de classifieurs. http://lheutte.free.fr/rdf/ArchiCombi.pdf, 2003. Consulté le 23 février 2026.

[19] IBM. What is bagging ? https://www.ibm.com/think/topics/bagging, 2024. Consulté le 23 février 2026.

[20] IBM. What is ensemble learning ? https://www.ibm.com/think/topics/ensemble-learning, 2024. Consulté le 23 février 2026.

[21] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm : A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.

[22] Albert Ko, Robert Sabourin, and Alceu de Souza Britto Jr. From dynamic classifier selection to dynamic ensemble selection. In *Proceedings of the 7th International Conference on Machine Learning and Applications*, 2007. Consulté le 23 février 2026.

[23] Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7 (NIPS 1994)*, pages 231–238, 1995. Article fondateur de l'identité de Krogh et Vedelsby sur l'ambiguïté des ensembles.

[24] Ludmila I. Kuncheva and Juan J. Rodríguez. A unified theory of diversity in ensemble learning. *Information Fusion*, 2023. Résumé disponible sur ResearchGate, consulté le 23 février 2026.

[25] G. Metzler. Machine learning td – méthodes ensemblistes. https://eric.univ-lyon2.fr/gmetzler/courses/contents/machinelearning/td4.pdf, 2022. Travaux dirigés, Université Lyon 2. Consulté le 23 février 2026.

[26] NS Academy. What is ensemble learning (bagging vs boosting vs stacking) ? https://medium.com/@sharetonschool/

what-is-ensemble-learning-bagging-vs-boosting-vs-stacking-c3c8cc94a3d5, 2023. Consulté le 23 février 2026.

[27] Robert E. Schapire. A brief introduction to boosting. *IJCAI*, 99 :1401–1406, 1999.

[28] scikit-learn-contrib. Deslib : A python library for dynamic classifier and ensemble selection. https://github.com/scikit-learn-contrib/DESlib, 2020. Consulté le 23 février 2026.

[29] Turing College. Ensemble learning : Key techniques explained. https://www.turingcollege.com/blog/ensemble-learning-2, 2024. Consulté le 23 février 2026.

[30] Ultralytics. Exploration de l'apprentissage d'ensemble : son rôle dans l'ia et la ml. https://www.ultralytics.com/fr/blog/exploring-ensemble-learning-and-its-role-in-ai-and-ml, 2024. Consulté le 23 février 2026.

[31] Wikipedia. Bias–variance tradeoff. https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff, 2026. Consulté le 23 février 2026.

[32] Wikipedia. Ensemble learning. https://en.wikipedia.org/wiki/Ensemble_learning, 2026. Consulté le 23 février 2026.

[33] Wikipédia. Théorème du jury. https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_du_jury, 2026. Consulté le 23 février 2026.

[34] Kaleemullah Younas. Bagging, boosting and stacking : Ensemble learning in ml models. https://medium.com/@kaleemullahyounas123/bagging-boosting-and-stacking-ensemble-learning-in-ml-models-8eb2f73afb07, 2023. Consulté le 23 février 2026.